CubeCAD: Unsupervised Deep Learning and Cuboid Abstraction for 3D CAD Reverse Engineering

Altay Kacan[1,3], Youssef Youssef[1,2], Bertram Fuchs[1], Oliver Lohse[1], Stefan Goetz[3], and Sandro Wartzack[3]

[1] Siemens AG, Munich, Germany
{altay.kacan, bertram.fuchs, oliver.lohse}@siemens.com
[2] Technical University of Munich, Munich, Germany
youssef.youssef@tum.de
[3] Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany
{goetz, wartzack}@mfk.fau.de

Abstract. Neutral exchange formats for 3D CAD such as STEP enable exporting and importing raw geometries between different CAD systems, but they do not preserve the model histories required for downstream modification. As a result, CAD experts often must rebuild editable 3D CAD models manually. This paper investigates whether unsupervised AI methods can automate parts of this process. A a novel coarse-to-fine unsupervised AI method for 3D CAD reverse engineering is proposed. The methodology first decomposes the input 3D geometry into cuboid primitives, recognizes symmetries, then predicts construction paths and sketch profiles for fine-grained reconstruction. Experiments on the ABC and ShapeNet datasets show that CubeCAD achieves higher reconstruction quality than prior unsupervised methods, with clear advantages in multi-category training. The results highlight limitations for mechanical parts and indicate that geometric reconstruction guided unsupervised training is insufficient for industry-grade 3D CAD reverse engineering.

Keywords: 3D CAD Reverse Engineering · Unsupervised Learning · Shape Abstraction.

1 Introduction

3D Computer-aided design (CAD) models are essential for modern product lifecycle management (PLM) and manufacturing. They enable engineers to create, visualize, and analyze complex 3D geometries with high precision [18, 20]. PLM systems and 3D modeling software such as Siemens NX integrate design data across multidisciplinary teams. Integration with computer-aided manufacturing (CAM) and computer-aided engineering (CAE) systems reduces time-to-market while improving product quality [5].

The model history, or feature tree, of 3D CAD models contains all construction steps used to build the final 3D geometry, enabling easy design modifications. This is fundamental to product development, where designs must be

iteratively refined to meet manufacturing and engineering requirements [5]. In feature-based modeling, CAD experts sketch 2D profiles and apply operations such as extrude, revolve, or sweep to generate 3D geometry, with any change automatically propagating through the entire model history [18].

Beyond the design process itself, customers, suppliers, and contract manufacturers rely on sharing 3D models to coordinate design specifications, manufacturing requirements, and assembly constraints. However, most commercial 3D CAD systems developed by different vendors are not natively interoperable. Each system uses proprietary file formats for the model history that cannot be exchanged directly between competing systems. Neutral exchange formats (STEP ISO 10303-242 [6]) are therefore required to transfer models across organizations that use different 3D CAD systems. These formats only describe the geometry of the product without the model history and are thus difficult to edit.

This limitation has direct consequences in practice. For example, contract manufacturers regularly receive STEP files from customers and must adapt the received geometry to their own manufacturing processes, tooling constraints, and simulation pipelines. Because the STEP file contains no model history, the contract manufacturer cannot modify the 3D model easily. Instead, the geometry must be partially or fully reconstructed *manually* in the recipient's 3D CAD system. Another relevant use case is 3D CAD system migration, where the parametrization and model histories from the old system usually cannot be imported into the new system.

The lack of publicly available industry-grade training data and the aforementioned challenges motivate the following research question: how can *unsupervised AI methods* be developed to *automate steps* of the manual 3D CAD reverse engineering process?

The rest of this paper is organized as follows: Section 2 reviews the state of the art in 3D CAD reverse engineering focusing on recent AI-based methods. Section 3 presents the proposed methodology. Section 4 reports experimental evaluation and validation. And finally, Section 5 summarizes the paper and outlines possible directions for future work.

2 State of the Art

This section reviews existing approaches to 3D CAD reverse engineering and identifies the research gap that motivates this paper.

2.1 AI Methods for 3D CAD Reverse Engineering

In feature-based modeling, CAD experts usually sketch a 2D profile and apply operations such as extrude, revolve, or sweep to form 3D geometry. Both supervised [19] and unsupervised [12, 16] AI methods have been proposed.

SfmCAD [11] is the first unsupervised AI-based method for 3D CAD reverse engineering to support CAD operations beyond extrusions. SfmCAD represents

extrude, loft, revolve, and sweep operations in a unified manner. Each operation is represented as a 2D sketch profile and a 3D sweep path. A two-stage coarse-to-fine strategy first predicts sweep paths, then refines local detail. All unsupervised feature-based AI methods reviewed here predict a *fixed number* of CAD operations per input 3D geometry. In practice, different parts require different numbers of design steps, and a fixed length limits the generalizability and practical utility of the proposed methods.

2.2 LLM-Based 3D CAD Reverse Engineering Methods

Large Language Models (LLMs) are transformer-based neural networks pretrained on large amounts of text, which can then be fine-tuned for specific downstream tasks. Most existing methods reformulate the CAD reverse engineering task as a code generation problem [9, 17]. Given a point cloud, the AI model generates executable code to reconstruct the input geometry. Usually, the open-source CadQuery library [1] is used. Despite their strengths, these approaches are limited to sketch-extrude operations and the model histories in CadQuery are not directly compatible with industry-standard 3D CAD systems.

2.3 Research Gap

Existing AI methods that use feature-based modeling are either supervised, restricted to CAD operations, or locked to a fixed number of construction steps. LLM-based methods require large datasets and intensive computational resources to train and use.

No existing method fully addresses the challenges encountered in real-world CAD reverse engineering. In particular, current supervised approaches rely on large datasets of annotated model histories which are not available for industry-grade CAD models. Unsupervised approaches rely on manually adapting the number of construction operations to part complexity, mostly do not support advanced CAD commands beyond simple extrusions, and do not aim to recover design intent through symmetry detection.

3 Methodology

CubeCAD is an unsupervised deep learning method for reverse engineering parametric 3D CAD models from raw 3D geometry. It works in a coarse-to-fine manner, starting with basic shapes and progressively refining geometric details. CubeCAD is trained using *unsupervised learning*, which requires no ground truth model histories. The first stage decomposes the input 3D geometry into a set of cuboid primitives that capture the overall structure. Unlike state-of-the-art methods such as SfmCAD [11] and SECAD-Net [12], which produce fixed-length sequences, CubeCAD generates a *dynamic amount of primitives*. In the second stage, *shape symmetry* detection identifies repetitions among the predicted cuboids. The third stage predicts sweep paths per predicted cuboid, and the final

stage synthesizes 2D sketches for the sweep operations. Similar to SfmCAD [11], the sweep operations are used to represent extrusions or revolves. This representation is designed to be translatable into commercial 3D CAD systems via future adapter implementations; however, this paper does not yet demonstrate a full end-to-end pipeline for importing into 3D CAD systems.

Each of the four stages is trained independently with its own loss functions, enabling modular optimization. The stages build on individual methods from the literature, specifically cuboid abstraction [21], symmetry detection [3], and sweep-based 3D CAD reconstruction [11] from raw 3D geometry. These methods are adapted and extended into a unified coarse-to-fine framework. The core novelty of this work is the integration of existing methods for unsupervised shape abstraction, symmetry detection, and CAD reconstruction in a unified pipeline that allows dynamic primitive counts and partial design intent recovery.

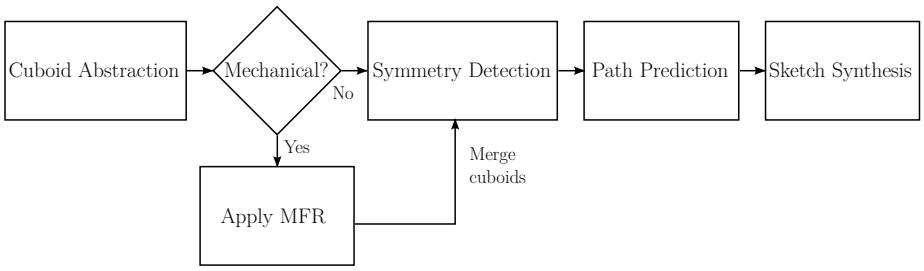


Fig. 1. Overall pipeline of CubeCAD with optional additional cuboids from Machining Feature Recognition (MFR) algorithms for mechanical parts.

3.1 Stage 1: Cuboid Abstraction

To unify the input 3D representation, point clouds are sampled on the surface of the B-Rep models or meshes. The first stage employs an unsupervised Variational Autoencoder (VAE) from Yang et al. [21] to partition the input point cloud into scaled cuboid primitives. The encoder maps the input geometry into a latent code, which a decoder transforms into oriented bounding boxes that capture the coarse volumetric structure. Each cuboid is represented by affine transformation parameters (rotation, translation, scale) and an existence probability that determines the number of primitives per shape. Training relies on unsupervised losses for ensuring accurate reconstructions and an appropriate number of primitives. This stage is adopted without modification from [21].

The method of Yang et al. [21] relies on pure geometric reconstruction. To improve performance in this stage, machining feature recognition (MFR) methods can be employed to predict additional cuboids for geometrical features specific to mechanical parts, such as holes, slots, or pockets. These cuboids can then be

merged with the VAE-predicted cuboid set before subsequent stages, improving performance especially for manufacturing-relevant geometric features.

3.2 Stage 2: Symmetry Detection

The second stage captures symmetries such as repetitions of the same primitive by decoupling shape geometry from spatial pose, adopting the ProGRIP framework [3]. Two transformer-based neural networks operate hierarchically first at the geometry level (how do the primitives look like) and the pose level (how are the primitives placed). Transformers are attention-based neural architectures that process sets of elements without assuming a fixed order, making it suitable for unordered geometric primitives.

At the *geometry level*, a PointNet encoder [14] extracts a global feature vector from the point cloud, which a geometry transformer processes to predict N canonical primitive representations. Each part k is decoded by a dedicated Multi-Layer Perceptron (MLP) into a geometry descriptor:

$$\mathcal{G}_k = (\mathbf{s}_k,\ \mathbf{z}_k), \quad k = 1, \ldots, N \tag{1}$$

where $\mathbf{s}_k \in \mathbb{R}^3$ defines the axis-aligned cuboid scale and $\mathbf{z}_k$ is a latent embedding vector capturing geometric shape. At the *pose level*, a pose transformer predicts M posed instances per primitive, each characterized by translation, quaternion rotation, and existence probability. For example, a table's four legs share one canonical primitive geometry but differ in placement. This two-level design produces $N \times M$ posed cuboids, matched to Stage 1 outputs via the Hungarian algorithm [10]. This stage uses N=10 canonical primitives and M=6 poses.

3.3 Stage 3: Construction Path Prediction

The third stage transforms abstract primitive embeddings into parametric sweep paths for 3D CAD operations. For each active primitive, a dedicated MLP outputs a 16-dimensional parameter vector following the Box+Path parametrization from Li et al. [11]. The training loss based on signed distance fields (SDFs) combines reconstruction error, smoothness regularization, and dimension regularization. The training of this stage is identical to the method introduced by Li et al. [11].

3.4 Stage 4: Sketch Synthesis

The final stage of CubeCAD generates 2D cross-section sketch profiles for the sweep operations to reconstruct fine details. Again, following Li et al. [11], this stage learns continuous SDFs for sketch profiles, where negative values indicate interior points and positive values indicate exterior points. This representation is well suited for neural network training as it enables gradient-based optimization. The predicted sketches are converted to 3D surfaces via differentiable sweep operations following Li et al. [11]. The individual primitive SDFs are combined

via a union operation to produce the final 3D occupancy prediction [11]. The neural network for Stage 4 is trained with mean squared error against ground truth occupancies.

3.5 3D CAD Model Construction

Before the sweep paths and sketch profiles can be imported into a real commercial 3D CAD system, the implicit SDF representations (which are continuous functions) must be converted into discrete curve geometry. Following [12], a contour extraction process is applied to the sketch of each primitive. It is possible to develop adapters can translate sketch profiles, sweep paths, and pose transformations into the native formats of target 3D CAD systems. This would allow CAD experts to modify individual features, adjust sketch profiles, and alter sweep trajectories within their familiar 3D CAD environment.

4 Experiments & Results

This section summarizes the datasets, metrics and experiments that were used for validating CubeCAD. Results on two datasets are presented. A discussion concludes the section.

4.1 Setup

CubeCAD is evaluated on two datasets. The ABC dataset [8], containing diverse mechanical CAD parts, serves as the primary dataset for quantitative evaluation. Following prior work [12, 22], approximately 5,000 models are used for training, while the same 50 models from SECAD-Net [12] are held out for per-shape fine-tuning and evaluation. A case study using a custom 3D CAD model is also presented.

The ShapeNet dataset [2] is included to enable direct comparison with prior unsupervised AI-based CAD reverse engineering methods. Three furniture categories (chairs, tables, sofas) are used for training with approximately 13,000 shapes in total. For evaluation, 40 models per category are chosen. Two experiment settings are defined: the single-category experiment trains exclusively on chairs and the multi-category experiment trains all three categories *jointly*.

Reconstruction quality is assessed using Chamfer Distance (CD) [4] and Normal Consistency (NC) [13]. CD measures the surface reconstruction quality and NC indicates the surface alignment. The number of primitives and the number of unique primitives are also reported. The difference reflects the effectiveness of the symmetry learning stage. All evaluations follow the protocol of SECAD-Net [12] and SfmCAD [11], with point clouds uniformly sampled from the surfaces of ground truth and reconstructed meshes.

The four stages of CubeCAD are trained independently and sequentially. CubeCAD without the addition of Machining Feature Recognition (MFR) is used for the experiments on the ABC and ShapeNet datasets. All experiments

are conducted on a single NVIDIA RTX A6000 GPU. After training on the whole dataset, per-shape fine-tuning adapts Stages 3 and 4 to individual test shapes while Stages 1 and 2 remain frozen. This process requires approximately 4 minutes per shape.

4.2 Results on Mechanical Parts

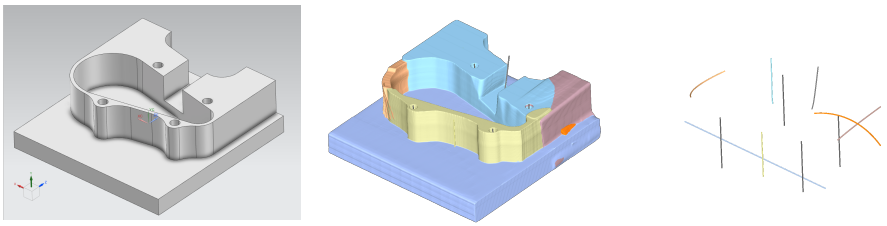
Table 1 shows the results of CubeCAD compared to existing unsupervised AI methods[4] on a subset of the ABC dataset [8] taken from [22]. CubeCAD achieves the best geometric reconstructions compared to prior work. SECAD-Net has the smallest number of primitives but the reconstruction quality is worse. The average number of primitives and number of unique primitives are very close to one another for CubeCAD. This suggests that CubeCAD does not learn strong symmetries on chosen subset of mechanical parts. This could be caused by the learned cuboid abstractions not being suitable to capture the shapes seen in the chosen subset of ABC dataset. Despite this shortcoming, the geometric reconstruction quality is higher than SECAD-Net. This is because CubeCAD is not limited to sketch-extrude operations and can capture finer details.

A case study using CubeCAD enriched with Machining Feature Recognition (MFR) is also carried out. Figure 2 illustrates the target geometry and the predicted sweep operations. CubeCAD recovers the overall geometric shape accurately and the cuboids from the MFR algorithm enable the reconstruction of holes and the slot geometry. It is observed that extensive post-processing steps are necessary to recover an exact B-Rep, especially due to the mismatch between the surfaces of neighboring sweeps (e.g. the surfaces of the yellow, orange, and light blue sweeps are not exactly level).

Table 1. Quantitative results on 3D CAD Models from the ABC [8] dataset of mechanical parts. Chamfer distance is multiplied by 10^3. Best values in bold.

Method	**CD** ($\times 10^3$) $\downarrow$	**NC** $\uparrow$	**# Prim.** $\downarrow$	**# Unique Prim.** $\downarrow$
UCSG-Net [7]	1.849	0.820	12.84	12.84
CSG-Stump [15]	4.031	0.828	17.18	17.18
ExtrudeNet [16]	0.471	0.852	14.46	14.46
SECAD-Net [12]	0.330	0.863	**4.30**	**4.30**
CubeCAD	**0.324**	**0.910**	5.04	4.90

[4] Values are taken from SECAD-Net [12] and the CubeCAD performance on the same subset of ABC parts are reported. SfmCAD is not included as the official code was not available and exact subset of the ABC dataset used for evaluation was not reported.



(a) (b) (c)

Fig. 2. Case study on a custom 3D CAD model using the CubeCAD variant with MFR. (a) Imported B-Rep body in Siemens NX without model history (b) Predicted primitives, primitives that remove volume (i.e. holes) are shown with gray paths (c) Predicted sweep paths. Best viewed in color.

4.3 Results on ShapeNet

To further validate CubeCAD, results on the ShapeNet [2] dataset are presented. The best state of the art method, SfmCAD is reimplemented and validated for correctness as the code, pretrained models, and exact evaluation data were not available during the development of this paper. Table 2 shows the results of the single category and the multi-category experiments. CubeCAD outperforms the reimplementation of SfmCAD under the current training and evaluation setup. The strengths of CubeCAD are more significant in the multi-category setting resulting in about 50% improvement in mean CD. It is hypothesized that since each category requires a different number of primitives and SfmCAD is inherently limited to a constant number of construction paths, it fails to reconstruct the 3D geometry accurately.

The difference of total and unique primitives indicates CubeCAD learns symmetries. Figure 3 shows the predicted construction paths for a chair from SfmCAD [11] and CubeCAD *in the multi-category setting*. The SfmCAD paths are entangled and do not reflect the inherent structure of the 3D shape unlike the reported results by Li et al. [11]. One potential reason is the multi-category setting. When trained only on chairs and with the manually determined correct number of primitives, SfmCAD can predict meaningful paths. However, the performance degrades when trained on shapes from multiple categories as illustrated in Figure 3. The cuboid abstraction stage of CubeCAD mitigates this issue. The predicted paths clearly follow the structure of the chair and symmetries (e.g. front legs and back legs, armrests) are recognized correctly as indicated by the color.

4.4 Discussion

The presented quantitative and qualitative analysis shows that CubeCAD can outperform previous state of the art unsupervised AI-based methods for 3D CAD reverse engineering. Furthermore, CubeCAD can generalize better to multiple

Table 2. Quantitative results on ShapeNet [2] dataset. All reported values are mean values across 40 shapes for single category (trained on only chairs) and 40 shapes per category in the multi-category experiments (trained on 3 categories). Chamfer distance is multiplied by 10^3. Best values in bold. *: Reimplementation.

Categories	Method	CD ($\times 10^3$) ↓	NC ↑	# Prim. ↓	# Unique Prim. ↓
Single	SfmCAD* [11]	0.704	0.839	10.00	10.00
	CubeCAD	**0.636**	**0.860**	**8.06**	**6.08**
Multi	SfmCAD* [11]	1.233	0.846	10.00	10.00
	CubeCAD	**0.565**	**0.882**	**8.56**	**6.92**

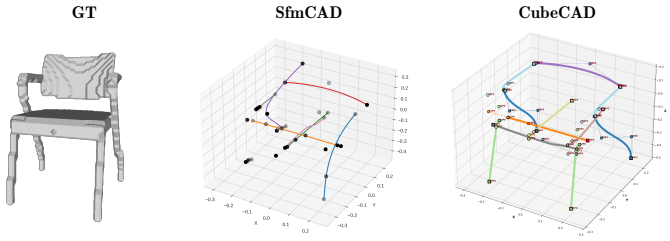


Fig. 3. Ground truth (GT) and qualitative results of the reimplemented SfmCAD [11] and CubeCAD in the *multi-category* setting. The path colors of the CubeCAD results indicate unique primitives, same colors mean repeated primitives.

categories compared to SfmCAD [11] and predict construction paths mirroring the inherent structure of the 3D objects as shown in the multi-category experiments on ShapeNet [2].

It is observed that the predicted construction paths and sketches are more meaningful for objects that have a clear hierarchical structure like chairs, tables or sofas compared to mechanical parts. Most shapes in the ABC dataset [8] cannot be broken down into abstract primitives as they are usually very simple and there is no hierarchical object structure unlike furniture. The case study confirms this observation for a more complex mechanical part. One possible reason for worse model history recovery for mechanical parts is the purely geometric reconstruction losses and the lack of real 3D CAD model histories during training.

Furthermore, it has been observed that extensive post-processing steps are necessary to get B-Rep models that are watertight and accurate for mechanical parts. Most previous works in unsupervised 3D CAD reverse engineering show only mesh or voxel reconstructions, especially of mechanical parts [11,12]. These challenges significantly limit the usability of such methods in industrial scenarios. This leads to the conclusion that alternative AI-based approaches that are trained with historical data of model histories instead of geometric reconstruction alone are a more promising research direction.

5 Conclusion & Future Work

This paper investigates the automation of 3D CAD reverse engineering through a novel unsupervised AI method, CubeCAD. The goal is to reduce the manual reconstruction of non-editable exchange formats, often done by contract manufacturers or when companies migrate 3D CAD systems. The presented results provide a partial but important answer to the formulated research question: unsupervised AI methods can partially automate steps of the 3D CAD reverse engineering problem, especially when the objects have inherent structures that repeat as is the case for furniture in ShapeNet. However, the experiments also make clear that this progress does not yet fully solve the industrial problem. In particular, the challenges of predicting abstractions for mechanical parts and the need for substantial post-processing to get industry-grade B-Reps indicate that unsupervised training with geometric reconstruction losses is not enough. The main contribution of this work is therefore both methodological and diagnostic, it advances the state of the art while clarifying what remains missing for 3D CAD reverse engineering in industrial and PLM contexts. Future work can include fine-tuning based on historical data to improve sweep path predictions and adapters to commercial 3D CAD systems.

Acknowledgments. This research has been funded by Siemens Foundational Technologies in Munich, Germany.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. CadQuery contributors: CadQuery. Zenodo (Oct 2025). https://doi.org/10.5281/zenodo.14590990
2. Chang, A.X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., et al.: Shapenet: An information-rich 3d model repository. arXiv preprint arXiv:1512.03012 (2015)
3. Deng, B., Kulal, S., Dong, Z., Deng, C., Tian, Y., Wu, J.: Unsupervised learning of shape programs with repeatable implicit parts. Advances in Neural Information Processing Systems **35**, 37837–37850 (2022)
4. Fan, H., Su, H., Guibas, L.J.: A point set generation network for 3d object reconstruction from a single image. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 605–613 (2017)
5. Groover, M., Zimmers, E.: CAD/CAM: computer-aided design and manufacturing. Pearson Education (1983)
6. ISO, I., et al.: Industrial automation systems and integration–product data representation and exchange–part 242: Application protocol: Managed model-based 3d engineering (2014)
7. Kania, K., Zieba, M., Kajdanowicz, T.: Ucsg-net-unsupervised discovering of constructive solid geometry tree. Advances in neural information processing systems **33**, 8776–8786 (2020)

8. Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., Panozzo, D.: Abc: A big cad model dataset for geometric deep learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 9601–9611 (2019)
9. Kolodiazhnyi, M., Tarasov, D., Zhemchuzhnikov, D., Nikulin, A., Zisman, I., Vorontsova, A., Konushin, A., Kurenkov, V., Rukhovich, D.: cadrille: Multi-modal cad reconstruction with reinforcement learning. In: The Fourteenth International Conference on Learning Representations (2025)
10. Kuhn, H.W.: The hungarian method for the assignment problem. Naval research logistics quarterly **2**(1-2), 83–97 (1955)
11. Li, P., Guo, J., Li, H., Benes, B., Yan, D.M.: Sfmcad: Unsupervised cad reconstruction by learning sketch-based feature modeling operations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4671–4680 (2024)
12. Li, P., Guo, J., Zhang, X., Yan, D.M.: Secad-net: Self-supervised cad reconstruction by learning sketch-extrude operations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 16816–16826 (2023)
13. Mescheder, L., Oechsle, M., Niemeyer, M., Nowozin, S., Geiger, A.: Occupancy networks: Learning 3d reconstruction in function space. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 4460–4470 (2019)
14. Qi, C.R., Su, H., Mo, K., Guibas, L.J.: Pointnet: Deep learning on point sets for 3d classification and segmentation. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 652–660 (2017)
15. Ren, D., Zheng, J., Cai, J., Li, J., Jiang, H., Cai, Z., Zhang, J., Pan, L., Zhang, M., Zhao, H., et al.: Csg-stump: A learning friendly csg-like representation for interpretable shape parsing. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 12478–12487 (2021)
16. Ren, D., Zheng, J., Cai, J., Li, J., Zhang, J.: Extrudenet: Unsupervised inverse sketch-and-extrude for shape parsing. In: European Conference on Computer Vision. pp. 482–498. Springer (2022)
17. Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., Aouada, D.: Cad-recode: Reverse engineering cad code from point clouds. arXiv preprint arXiv:2412.14042 (2024)
18. Shah, J.J., Mäntylä, M.: Parametric and feature-based CAD/CAM: concepts, techniques, and applications. John Wiley & Sons (1995)
19. Uy, M.A., Chang, Y.Y., Sung, M., Goel, P., Lambourne, J.G., Birdal, T., Guibas, L.J.: Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 11850–11860 (2022)
20. Weiler, K.J.: Topological Structures for Geometric Modeling (Boundary Representation, Manifold, Radial Edge Structure). Rensselaer Polytechnic Institute (1986)
21. Yang, K., Chen, X.: Unsupervised learning for cuboid shape abstraction via joint segmentation from point clouds. ACM Transactions On Graphics (TOG) **40**(4), 1–11 (2021)
22. Yu, F., Chen, Z., Li, M., Sanghi, A., Shayani, H., Mahdavi-Amiri, A., Zhang, H.: Capri-net: Learning compact cad shapes with adaptive primitive assembly. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 11768–11778 (2022)